ASSESSMENT TOOL-3

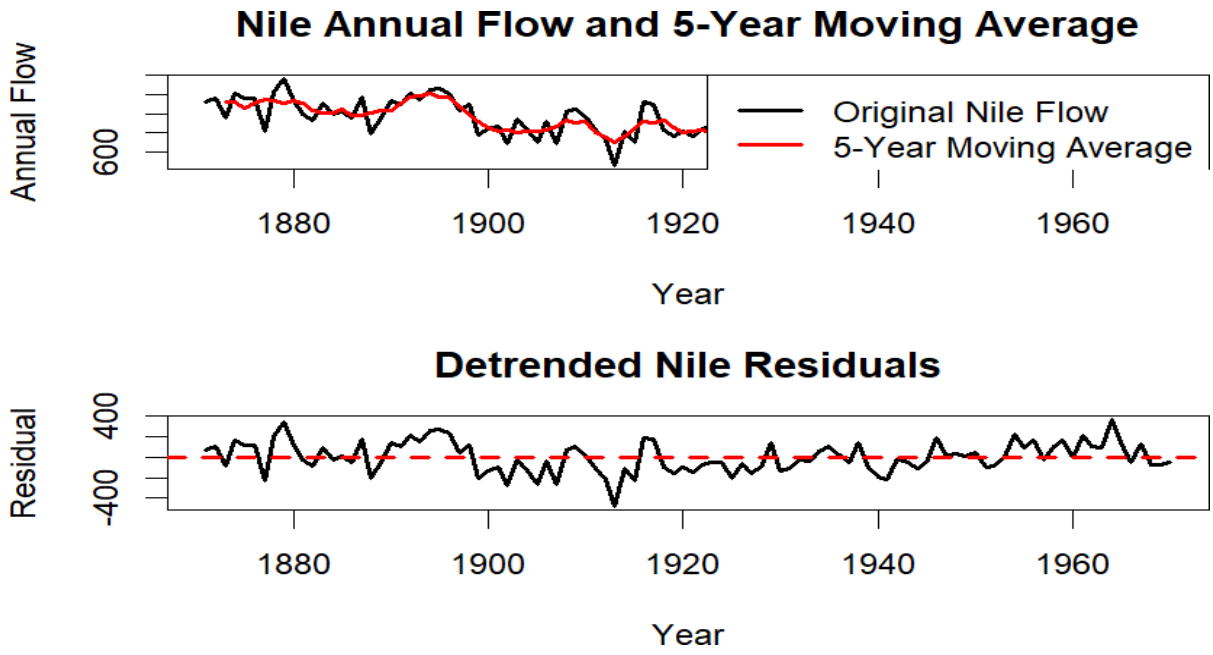
Time-Series & Geospatial Analysis Report

Code of Conduct:

I ***C Nithisha(192424375)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: **C Nithisha**

Question 1: Individual Time Series — Smoothing & Detrending



Code:

```
rm(list = ls())
data(Nile)
year <- 1871:1970
flow <- as.numeric(Nile)
moving_average <- stats::filter(
flow,
  rep(1/5, 5),
  sides = 2)
trend_model <- lm(flow ~ year)
```

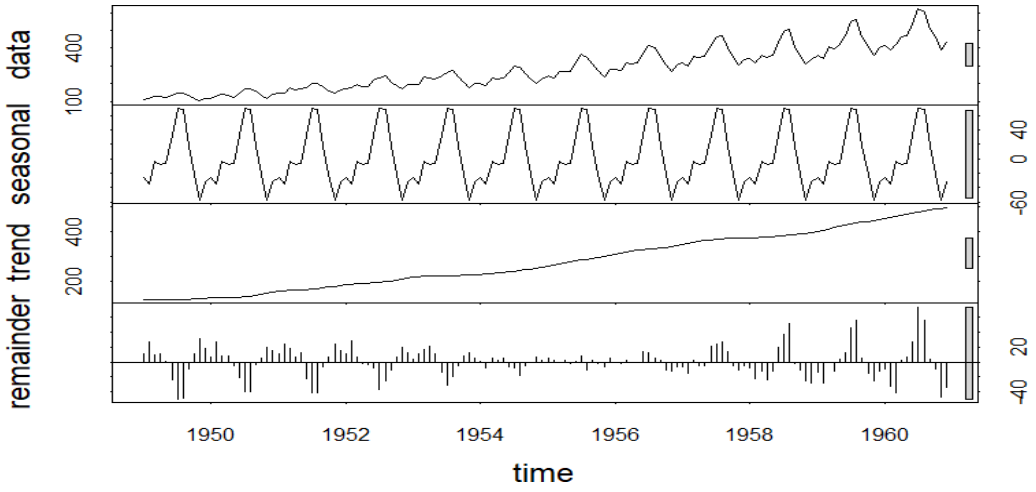
```
detrended_residuals <- residuals(trend_model)
```

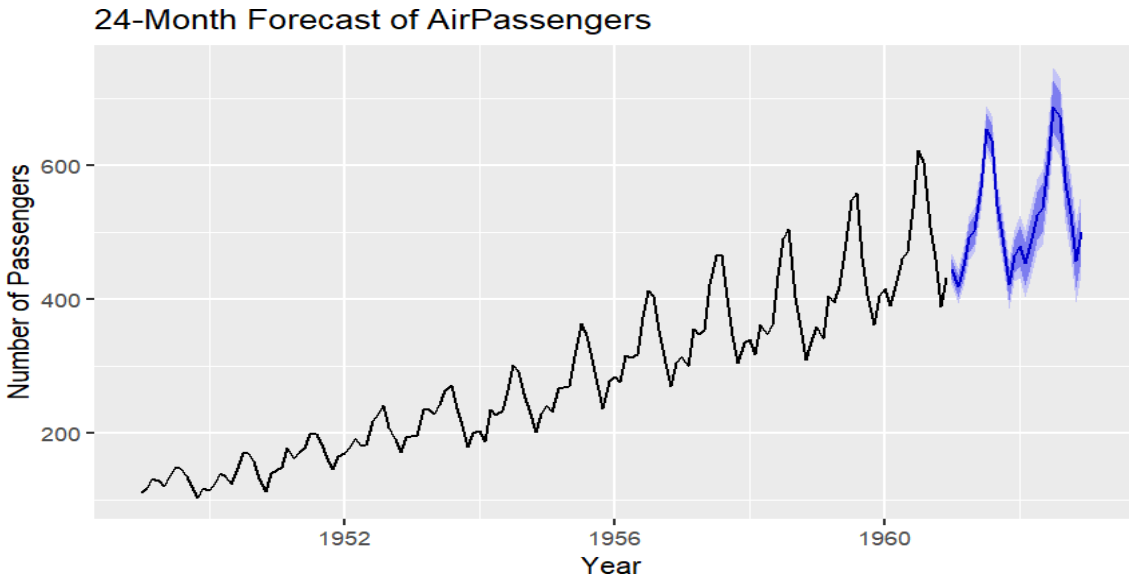
```
par(mfrow = c(2, 1))
graphics::plot.default(
year,
  flow,
  type = "l",
  lwd = 2,
  xlab = "Year",
  ylab = "Annual Flow",
  main = "Nile Annual Flow and 5-Year Moving Average")
graphics::lines(
  year,
  moving_average,
  col = "red",
  lwd = 2)
graphics::legend(
"topright",
  legend = c("Original Nile Flow", "5-Year Moving Average"),
  lty = 1,
  lwd = 2,
  col = c("black", "red") )
graphics::plot.default(
year,
detrended_residuals,
type = "l",
  lwd = 2,
  xlab = "Year",
  ylab = "Residual",
  main = "Detrended Nile Residuals")
graphics::abline(
  h = 0,
  col = "red",
  lty = 2,
  lwd = 2)
par(mfrow = c(1, 1))
```

Observation

The smoothed series shows a noticeable decrease in the Nile's average flow around the late 1890s, suggesting a possible level shift. The detrended residual plot also shows a sustained change around this period, indicating that the change is not simply explained by a linear trend.

Question 2: Seasonal Decomposition (STL) & Forecasting





Code:

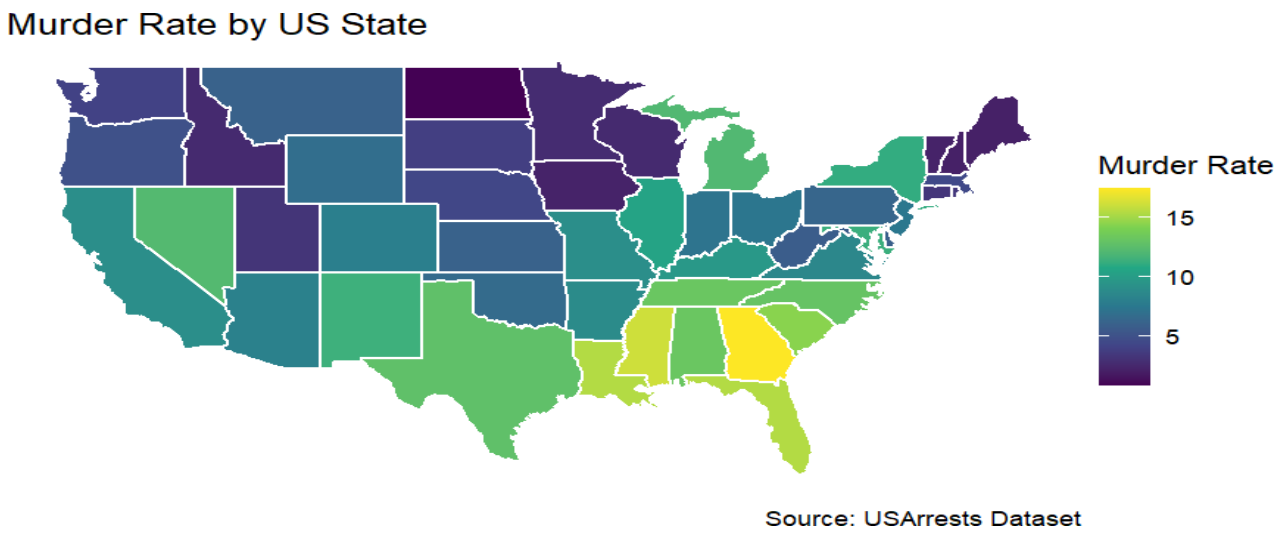
```
install.packages("forecast")
library(forecast)
```

```
air_ts <- ts(AirPassengers,
             start = c(1949, 1),
             frequency = 12)
air_stl <- stl(air_ts, s.window = "periodic")
plot(air_stl)
model <- auto.arima(air_ts)
forecast_result <- forecast(model, h = 24)
autoplot(forecast_result) +
  ggtitle("24-Month Forecast of AirPassengers") +
  xlab("Year") +
  ylab("Number of Passengers")
```

Observation

A **multiplicative decomposition** is more appropriate because the size of the seasonal fluctuations increases as the number of passengers increases. The forecast continues a repeating seasonal pattern with regular peaks and low points, consistent with the historical data.

Question 3 – Part A: Choropleth Map of US Murder Rates



CODE:

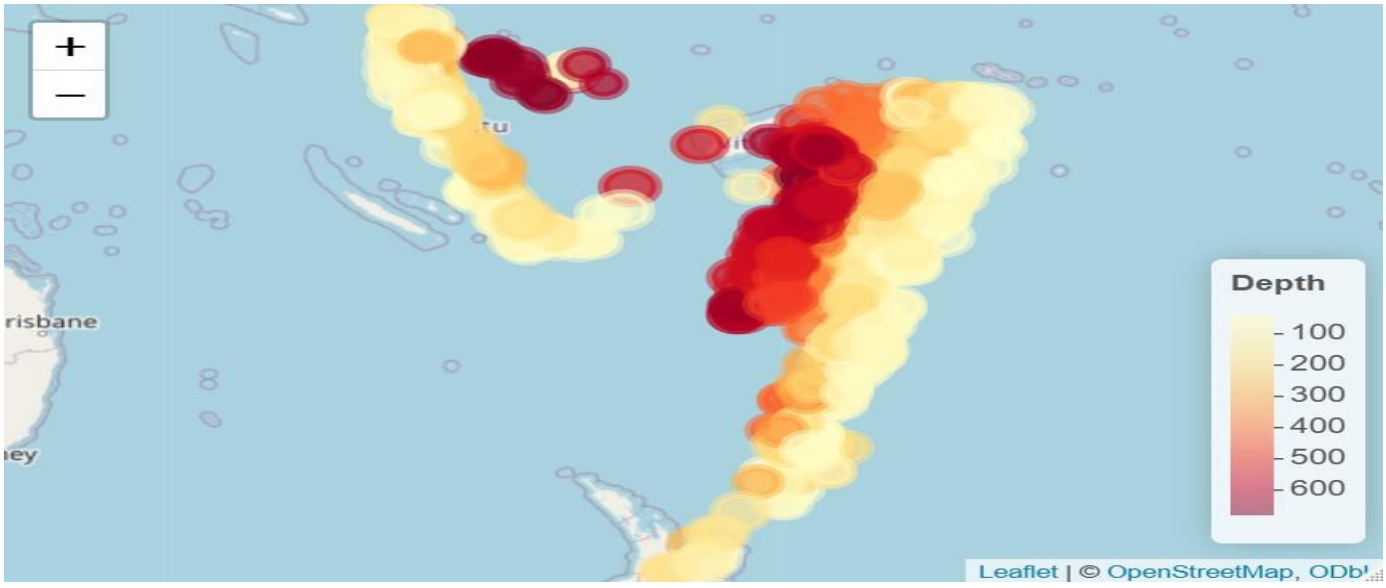
```
library(ggplot2)
library(maps)
library(dplyr)
arrests <- USArrests
arrests$state <- tolower(rownames(arrests))
states_map <- map_data("state")
```

```
map_data_joined <- left_join(
  states_map,
  arrests,
  by = c("region" = "state"))
ggplot(map_data_joined,
       aes(x = long,
           y = lat,
           group = group,
           fill = Murder)) +
  geom_polygon(color = "white") +
  coord_fixed(1.3) +
  scale_fill_viridis_c() +
  labs(
    title = "Murder Rate by US State",
    fill = "Murder Rate",
    caption = "Source: USArrests Dataset"
  ) +
  theme_void()
```

Observation

The output identifies the state with the highest murder rate. The map generally shows higher murder rates concentrated in several southern states.

Question 3 – Part B: Interactive Earthquake Map



CODE:

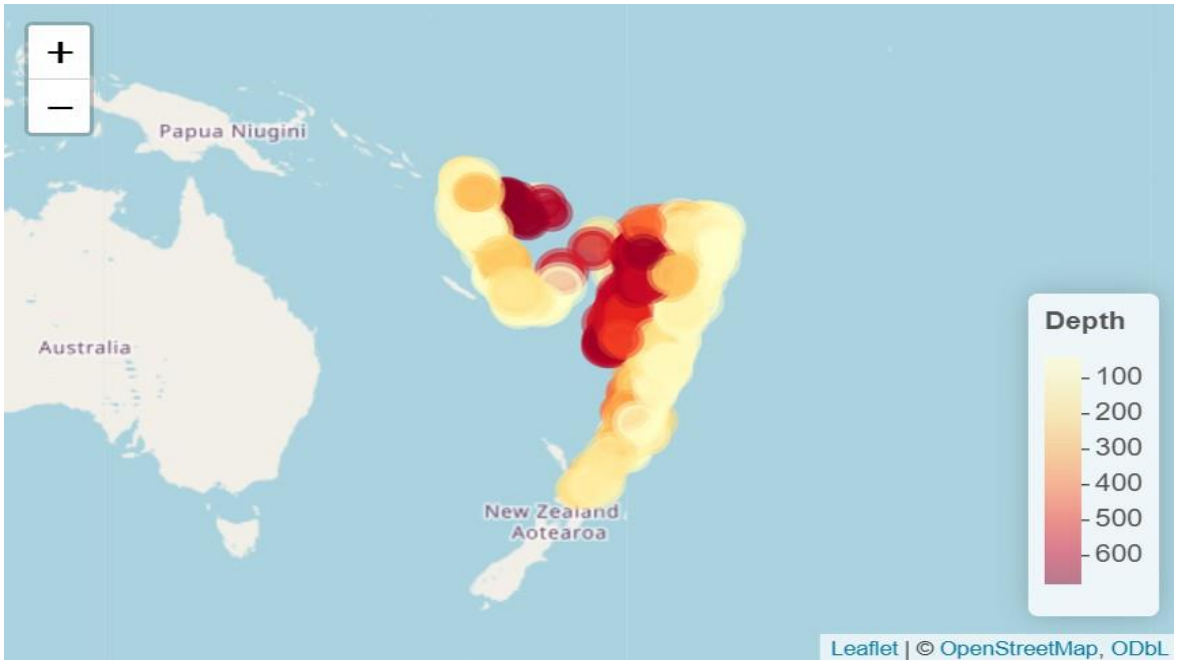
```
install.packages("leaflet")
library(leaflet)
```

```
earthquakes <- quakes
 pal <- colorNumeric(
  palette = "YlOrRd",
   domain = earthquakes$depth
)
leaflet(earthquakes) %>%
  addTiles() %>%
  addCircleMarkers(
    lng = ~long,
    lat = ~lat,
    radius = ~mag * 2,
    color = ~pal(depth),
    fillOpacity = 0.7,
    popup = ~paste(
    "Magnitude:", mag,
    "<br>Depth:", depth
    )
  ) %>%
  addLegend(
    position = "bottomright",
    pal = pal,
    values = ~depth,
    title = "Depth"
  )
```

Observation

The earthquakes are spatially clustered mainly around the Fiji–Tonga region. Larger earthquakes can also be examined interactively using the marker size and popup information.

Question 4: Cartogram with Map Projections



CODE:

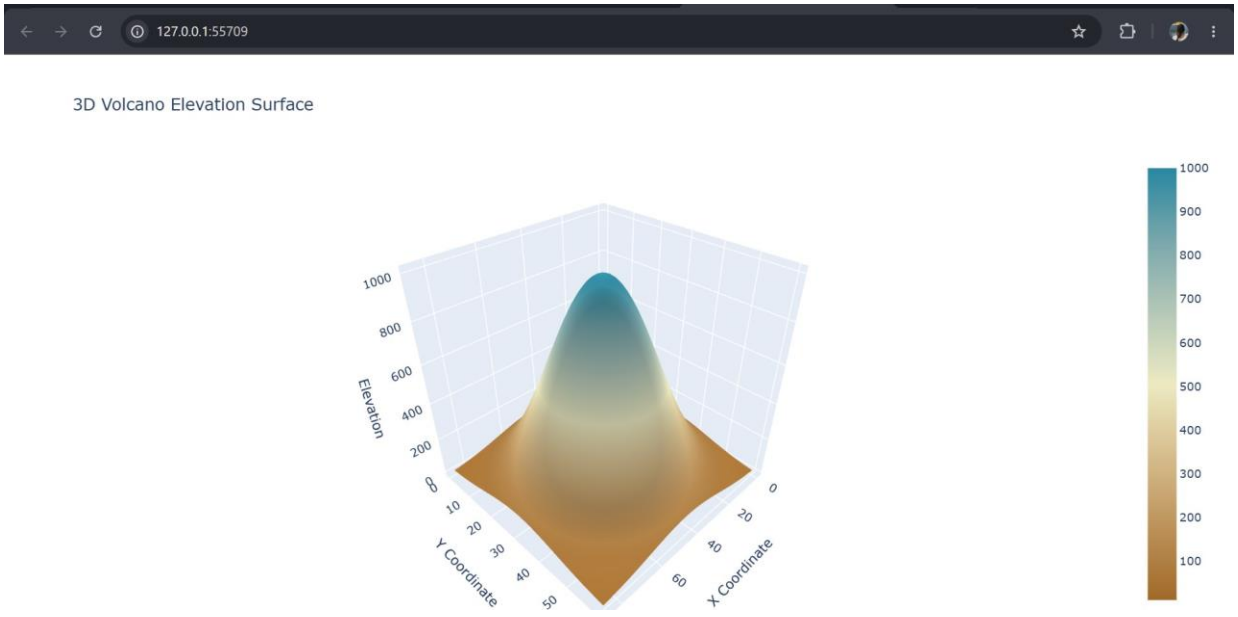
```
library(sf)
library(maps)
library(cartogram)
library(dplyr)
library(ggplot2)
states_data <- as.data.frame(state.x77)
states_data$state <- tolower(rownames(states_data))
head(states_data)
us_map <- maps::map(
  "state",
  plot = FALSE,
  fill = TRUE)
us_sf <- sf::st_as_sf(us_map)
# Create state name column
us_sf$state <- tolower(us_sf$ID)
us_sf <- dplyr::left_join(
  us_sf,
  states_data,
  by = "state")
us_sf <- us_sf[!is.na(us_sf$Population), ]
us_sf <- sf::st_make_valid(us_sf)
p1 <- ggplot(us_sf) +
  geom_sf(
    aes(fill = Population),
    color = "white"
```

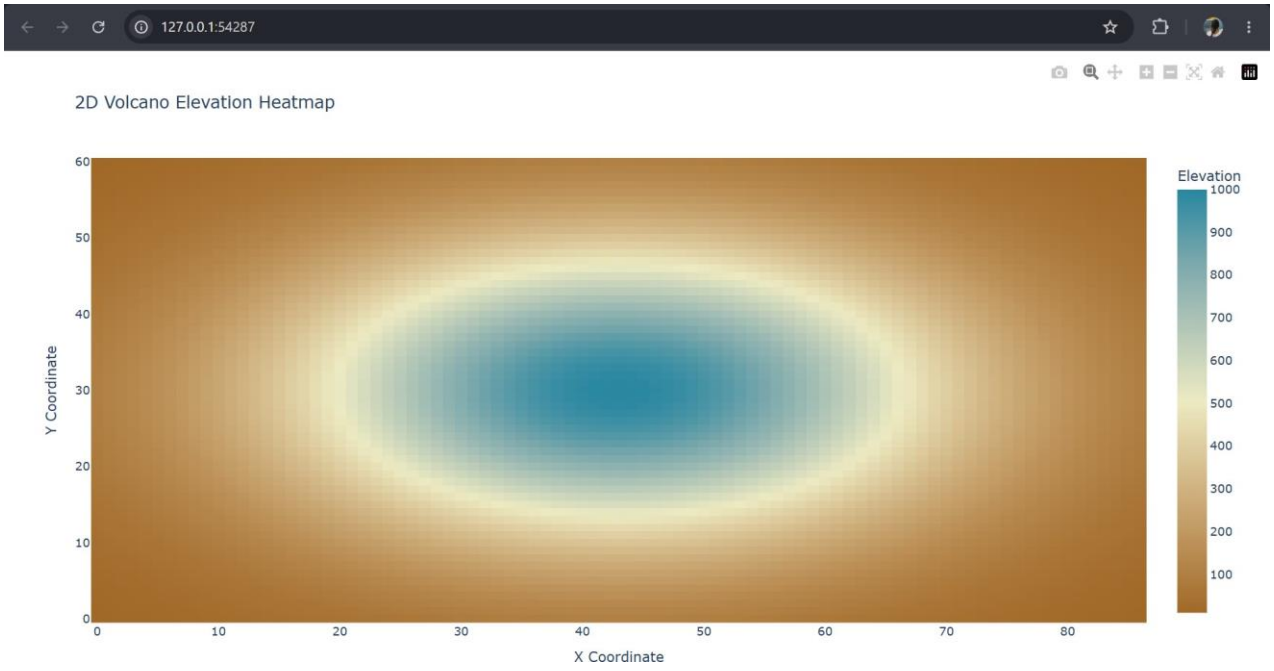
```
  ) +
  scale_fill_viridis_c() +
  ggtitle("Standard US Population Map") +
  labs(fill = "Population") +
  theme_minimal()
print(p1)
p2 <- ggplot(us_sf) +
  geom_sf(
    aes(fill = Population),
    color = "white"
  ) +
  coord_sf(crs = 5070) +
  scale_fill_viridis_c() +
  ggtitle("US Population Map - Albers Projection") +
  labs(fill = "Population") +
  theme_minimal()
print(p2)
us_projected <- sf::st_transform(
  us_sf,
  crs = 5070)
cartogram_map <- cartogram::cartogram_cont(
  us_projected,
  weight = "Population",
  itermax = 5)
p3 <- ggplot(cartogram_map) +
  geom_sf(
    aes(fill = Population),
    color = "white"
  ) +
  scale_fill_viridis_c() +
  ggtitle("Population-Weighted US Cartogram") +
  labs(fill = "Population") +
  theme_minimal()
print(p3)
```

Observation

In the cartogram, highly populated states become visually larger, while states with smaller populations shrink. This changes the visual importance of states compared with a normal geographic map, where physical land area determines their size.

Question 5: 3D Volcano Elevation Heatmap — Python





CODE:

```
import numpy as np
```

```
import pandas as pd
import plotly.graph_objects as go
x = np.arange(87)
y = np.arange(61)
X, Y = np.meshgrid(x, y)
Z = 1000 * np.exp(
    -(((X - 43) ** 2) / 800 + ((Y - 30) ** 2) / 400)
)
data = pd.DataFrame({
    "x": X.flatten(),
    "y": Y.flatten(),
    "elevation": Z.flatten()
})
print(data.head())
elevation_grid = data.pivot(
    index="y",
    columns="x",
    values="elevation")
fig_3d = go.Figure(
    data=[
        go.Surface(
            z=elevation_grid.values,
            colorscale="Earth")])
fig_3d.update_layout(
```

```
    title="3D Volcano Elevation Surface",
    scene=dict(
        xaxis_title="X Coordinate",
        yaxis_title="Y Coordinate",
        zaxis_title="Elevation"))
fig_3d.show()
```

Observation

The 3D surface plot clearly shows the volcano's height, slopes, and central peak, making its physical shape easier to understand. The 2D heatmap shows elevation differences using color but does not display the vertical structure as clearly.